AI Compiler Performance Engineering

Auto-Loops Books — ch03

Contents

1 AI Hardware Architecture and Compiler Constraints **1**

1.1 Hardware landscape 1
1.2 Constraint matrix 3
1.3 GPU constraints 5
1.4 CPU constraints 6
1.5 NPU constraints 7
1.6 Hardware-aware compilation 9
1.7 Benchmark methodology 10
1.8 YiRage hardware modeling 11
1.9 Key Takeaways 12
1.10 Conclusion 13

Bibliography **14**

Index **16**

Chapter 1

AI Hardware Architecture and Compiler Constraints

Chapters 1 and 2 argued that decode is bound by movement and orchestration, not arithmetic, and that the cure is a compiler that respects the hardware. This chapter supplies the hardware facts that compiler must respect—the concrete memory sizes, parallelism grains, and scheduling models of GPUs, CPUs, and NPUs—because every later pass is, in the end, a function of these constraints (NVIDIA, 2022; AMD, 2024a; Chen, 2020). The decode anchors from Chapter 1 set the stakes: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and removing host overhead bought 1.259× on a Qwen-2.5-7B decode cell without touching the math (Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024). The mistake to avoid is reasoning about hardware in the abstract. "The GPU has fast memory" is useless; "an H100 SM has 228 KB of shared memory and 256 KB of registers" is a constraint a tiling pass can solve against (NVIDIA, 2022). This chapter is deliberately concrete, because the difference between a decode kernel that fits and one that spills is measured in kilobytes on a specific SKU (SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

1.1 Hardware landscape

The AI hardware landscape is no longer GPU-only, and a compiler that targets only **nvidia** silicon is already behind production reality. The classes that matter for decode divide by execution model. An **nvidia** GPU is a SIMT machine with tensor cores, dynamically scheduling warps over a deep memory hierarchy; an

Table 1.1: AI accelerator classes and primary execution models.

Class	Execution model	Decode fit
NVIDIA GPU	SIMT + tensor cores	Graphs, fusion
AMD GPU	Wavefront + matrix cores	HIP fusion
x86/ARM CPU	Cache + SIMD	Coarse fusion
Edge NPU	Spatial dataflow	Static MegaKernel

amd GPU is similar in spirit, with wavefronts and matrix cores reached through HIP/ROCm; an **x86** or **arm** CPU executes cache-resident SIMD over a coherent memory system; and an edge **npu** such as AMD XDNA is a spatial-dataflow fabric of compute tiles with explicit local SRAM and no hardware cache (NVIDIA, 2022; AMD, 2024b,a). These are not variations on one machine—they have genuinely different residency, parallelism, and scheduling models, which is why one kernel cannot be optimal across them.

The reason the landscape matters for decode specifically is that each class fails differently at batch-1. A GPU is fast but pays a launch tax per operator, so it rewards fusion and graph capture; a CPU has modest bandwidth but flexible control, so it rewards cache-blocked coarse fusion; an **npu** has deterministic on-chip SRAM but no dynamic scheduling, so it rewards a statically-planned MegaKernel (Vellaisamy *et al.*, 2026; Chen, 2026; Mirage Project contributors, 2025). Reading Table 1.1, the "decode fit" column is really a prescription for what kind of compiled artifact each class wants, a theme the rest of the book develops. The entries are not aspirational: a captured fused kernel on the GPU, a cache-blocked bundle on the CPU, and a static MegaKernel on the NPU are each the artifact that the demonstrated decode wins of later chapters actually take on that class (Mirage Project contributors, 2025; Rotem *et al.*, 2018; Chen, 2026).

There is a historical reason the landscape fragmented, and it matters for compiler design. For a decade the industry optimized one architecture—the server GPU—and vendor libraries like cuDNN and cuBLAS absorbed the per-operator tuning, so a framework could treat the GPU as a fixed target (Chen *et al.*, 2018; Chen, 2020). That model broke as inference moved to phones, laptops, and custom accelerators, each with a different memory and parallelism structure, and as the cost of LLM serving made the launch and bandwidth overheads of the operator-library approach unacceptable (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The compiler's job grew from "generate a good GPU kernel" to "generate a good kernel for whichever of several incompatible machines this model deploys on," which is precisely why hardware modeling became central rather than incidental.

Table 1.2: Cross-hardware constraint matrix.

Dimension	GPU	CPU	NPU
On-chip SRAM	Shared mem / L1	L1/L2/L3	Tile buffers
Parallel grain	Warp	Core + SIMD	Spatial PE array
Dynamic shapes	Flexible	Flexible	Restricted

It is worth being concrete about how different the classes really are, because the differences are not matters of degree. A GPU schedules tens of thousands of threads dynamically and tolerates latency by oversubscription; a CPU runs a few dozen threads and tolerates latency by caching and prefetch; an NPU runs a fixed spatial array with no dynamic scheduling at all (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b). These are different computational models, not different speeds, and a compiler abstraction that papers over the difference—pretending an NPU is just a slow GPU—produces schedules that are illegal or pathological on the target. The landscape table is therefore a taxonomy of execution models the compiler must genuinely understand, not a performance ranking.

Multi-hardware delta. The same decoder layer, compiled for each class, produces a different legal schedule: a captured, fused megakernel on the **nvidia** GPU; a cache-blocked AOT bundle on the **x86**/**arm** CPU; a static tile-and-DMA plan on the **npu** (SemiAnalysis, 2024; AMD, 2024a; Rotem *et al.*, 2018). Identical math, three different bytes/token outcomes, which is the central reason this book treats hardware as a first-class compiler input rather than a deployment afterthought (Chen and YiRage Contributors, 2026).

1.2 Constraint matrix

Reducing the landscape to a **constraint matrix** is how a compiler reasons about it: each hardware class is a set of constraints along a few dimensions—on-chip capacity, **memory hierarchy** structure, **parallel** grain, and shape flexibility—that the compiler must satisfy simultaneously. Table 1.2 captures the axes. On-chip SRAM is programmer-managed shared memory on a GPU, hardware-managed cache on a CPU, and explicit tile buffers on an NPU; the **parallel** grain is the warp, the core-plus-SIMD lane, and the spatial PE array respectively; and dynamic shapes are flexible on GPU and CPU but restricted on a statically-configured NPU (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b).

The dimension that dominates decode is **bandwidth** against the **memory hierarchy**. Because batch-1 decode re-reads the full weight set and growing KV cache every token, the achievable token rate is bounded by how fast bytes move through the **memory hierarchy**, not by peak FLOPs (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The production surprise is rarely insufficient compute; it is illegal tier placement—a tensor that should have stayed in fast SRAM spilling to HBM or DDR and paying the **bandwidth** tax every token (Chen, 2026). The constraint matrix exists so the compiler can check, before codegen, that each tensor's residency fits the target tier rather than discovering the spill in p99 telemetry (Davies *et al.*, 2025; SemiAnalysis, 2024). Framed this way, the matrix is the interface between the hardware facts of this chapter and the compiler passes of the rest of the book: each pass reads the relevant row, and a change that violates a cell is rejected before it can become a runtime regression (Chen and YiRage Contributors, 2026; Chen, 2026).

The shape-flexibility row of Table 1.2 is easy to overlook but decisive for LLM serving. GPUs and CPUs handle dynamic shapes—variable sequence lengths, growing KV caches—naturally, recompiling or branching as needed; a statically-configured NPU does not, so deploying an LLM with its inherently dynamic decode loop to an NPU requires the compiler to bucket or pad shapes to a fixed set the static schedule can handle (AMD, 2024b; Lai *et al.*, 2023). This is why dynamic-shape support, which Relax made a first-class concern, is a real constraint and not a convenience: the **memory hierarchy** and **parallel** grain determine raw speed, but shape flexibility determines whether the workload can run on the target at all without expensive padding.

The matrix also clarifies why "just use the fastest hardware" is the wrong framing. The fastest raw **bandwidth** belongs to the datacenter GPU, but it carries the launch tax and the highest cost; the CPU has modest **bandwidth** but flexible control and low cost at low load; the NPU has deterministic on-chip **bandwidth** and the best power efficiency but the least flexibility (Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a). Each row of the matrix is a trade, and the right target depends on which constraints the deployment actually binds against—which is the routing decision Chapter 24 makes at fleet scale, grounded in exactly these per-class constraints.

Review gate. The operational discipline is to attach a roofline or buffer-accounting analysis for each fused region on the target SKU before merge, treating an over-budget region as a build error rather than a performance bug (Chen, 2026; Chen and YiRage Contributors, 2026). The constraint matrix is what makes that

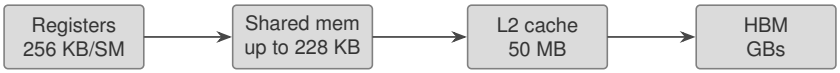


Figure 1.1: GPU memory hierarchy constraints for decode megakernels (H100 sizes).

gate mechanical: it gives the compiler the numbers to compare a region's footprint against (NVIDIA, 2022).

1.3 GPU constraints

The GPU's constraints are precise and small, and a decode kernel is solved against their exact values. On an H100 SM, the **shared memory** is configurable up to 228 KB (227 KB usable per block after 1 KB reserved), the register file is 65,536 32-bit registers (256 KB, capped at 255 per thread), and a 50 MB GPU-wide L2 backs them before HBM (NVIDIA, 2022). The **warp** of 32 threads is the scheduling grain, four warps form a warpgroup that issues the high-throughput WGMMA **tensor core** instruction, and Hopper adds thread-block clusters that share **shared memory** across CTAs (Shah *et al.*, 2024; NVIDIA, 2022). These numbers are the constraints Chapters 6–8 tile against.

The defining GPU trade is register pressure versus occupancy. Because resident threads share the 65,536-register file, a kernel using R registers per thread caps resident threads near $65536/R$, so a heavily fused decode kernel that keeps many partial sums in registers trades occupancy for residency (NVIDIA, 2022). At batch-1 that trade often favors residency—there is little parallelism to hide latency anyway—up to the spill threshold, beyond which a thread's registers spill to HBM-backed local memory and the win is lost (Vellaisamy *et al.*, 2026; Chen, 2026). The **tensor core** adds a second constraint: WGMMA wants operands in a specific layout, so the schedule must feed it from **shared memory** in the right shape or pay a conversion (Shah *et al.*, 2024).

The **shared memory** bank structure is a third constraint that decode kernels routinely trip over. H100 shared memory is 32 banks of 4 bytes, with consecutive words mapped to consecutive banks, so a **warp** accessing 32 addresses in distinct banks completes in one transaction while a pattern that collides on a bank serializes (NVIDIA, 2022). A tile placed in **shared memory** with a stride that aliases banks turns a fast access into a serial one, and because a decode kernel runs thousands of times per sequence the penalty compounds. The layout of a resident tensor—its padding and stride—is therefore part of the residency decision, not a detail, and a hardware-aware compiler chooses it against the bank count.

The launch tax is the GPU constraint most specific to decode. Every kernel the host launches pays a dispatch cost, and at batch-1 there is no batch to amortize it, so a decoder layer left as a dozen separate kernels pays that cost a dozen times per token (Vellaisamy *et al.*, 2026; Chen, 2026). This is why the GPU rewards fusion and graph capture so heavily: collapsing the layer into one launch, or capturing the launch sequence so the host stops re-dispatching, attacks the orchestration limiter directly (NVIDIA, 2024; Mirage Project contributors, 2025). The GPU's flexibility—dynamic scheduling, deep pipelines—is thus both its strength and the source of its dominant decode cost.

Multi-hardware delta. A Hopper GPU can hide latency with asynchronous TMA copies into **shared memory** and deep pipelines, a flexibility a static NPU lacks; but that same dynamism is why the GPU pays a launch tax the NPU does not, so the GPU's constraint is orchestration while the NPU's is capacity (SemiAnalysis, 2024; AMD, 2024b). The compiler must encode both, because the optimal schedule differs accordingly (Chen and YiRage Contributors, 2026).

1.4 CPU constraints

The CPU trades the GPU's raw bandwidth for flexible control and a deep, hardware-managed **cache** hierarchy, and its constraints are correspondingly different. Residency on a CPU is not allocated explicitly into a scratchpad; it is controlled indirectly by blocking the computation so each tile's working set fits and is reused within an L1, L2, or L3 **cache** level before eviction (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). The parallelism grain is the core plus its **simd** vector unit (AVX-512 on x86, NEON on ARM), so a decode kernel must vectorize across the lane width and tile across cores, a coarser grain than a GPU warp (Goodfellow *et al.*, 2016).

The CPU's signature hazard is **numa**. On multi-socket or multi-die servers, memory is partitioned across nodes, and a thread reading memory on a remote **numa** node pays a latency and bandwidth penalty that, for a memory-bound decode loop re-reading weights every token, can halve effective throughput (Davies *et al.*, 2025; Chen, 2026). The residency plan therefore includes thread and memory affinity—pinning decode threads to the **numa** node owning their weights, and replicating read-only weights per node when the bandwidth saving exceeds the memory cost. A second subtlety is that the hardware prefetcher builds an implicit pipeline for sequential access, so the highest-leverage CPU move is often laying out the weight stream contiguously so the prefetcher tracks it, rather than hand-rolling explicit movement (Goodfellow *et al.*, 2016).

The bandwidth arithmetic makes the CPU's decode ceiling concrete. At batch-1 the model re-reads its full weight set every token, so a 7B model in BF16 must move on the order of 14 GB per token from wherever the weights live; divided by a CPU's memory bandwidth, that sets a hard floor on token latency no compute optimization can beat (Kwon *et al.*, 2023; Chen, 2026). The CPU's advantage is not throughput but flexibility and cost: it handles dynamic shapes and control flow naturally, and it is economical at low query rates where a GPU would sit underutilized. A compiler targeting a CPU therefore optimizes for the regimes where that flexibility matters—variable-length, low-QPS, cost-sensitive endpoints—rather than trying to match GPU decode latency it structurally cannot reach.

A further CPU subtlety is **simd** width and its interaction with quantization. Wider vector units process more elements per instruction, but only if the data is laid out and aligned for them, and low-precision formats pack more elements per vector, so the **cache** footprint and the **simd** efficiency are coupled (Goodfellow *et al.*, 2016). A decode kernel that quantizes weights to int8 both shrinks the **cache** working set and fills the **simd** lanes more densely, a double win on a bandwidth-bound CPU—the same residency logic as the GPU, expressed through the **cache** and vector unit rather than shared memory.

Review gate. For a CPU target the gate is a **cache**-miss and bandwidth measurement at the decode operating point, because CPU residency is statistical (governed by the **cache** and prefetcher) rather than a static capacity that can be checked by arithmetic alone (Goodfellow *et al.*, 2016; Chen, 2026). A blocking plan that looks correct on paper must be validated against measured miss rates.

1.5 NPU constraints

The spatial **npu** is the most constrained and the most deterministic of the three, and its constraints must be satisfied entirely at compile time. On the AMD XDNA2/AIE-ML fabric, each compute tile has 64 KB of L1 **static** SRAM (eight 8 KB banks) with a dedicated DMA engine, each column adds a 512 KB L2 memory tile, and shim tiles bridge to off-chip DDR (AMD, 2024b,a). Crucially there is no hardware cache: what is resident is exactly what the compiler's DMA schedule placed, so a tensor that does not fit the 64 KB tile must be split across tiles or staged through L2, and the DMA chain to do so is part of the program, not a runtime fallback.

This makes **npu** compilation unforgiving but powerful. Unforgiving, because a mis-sized tile is a compile-time failure to fit rather than a slow path; powerful,

because once the **static** plan fits there is no cache-miss variance and no allocator, so the per-token cost is fully deterministic—ideal for the tight latency and power budgets of **edge** deployment (AMD, 2024a; Rotem *et al.*, 2018). The canonical **edge** decode fusion maps the attention chain as a spatial sweep across tiles, keeping intermediate scores in on-chip SRAM and streaming only inputs and outputs through DDR, the spatial analogue of a GPU megakernel (Dao *et al.*, 2022; Chen, 2026).

The spatial structure gives the **npu** a property neither GPU nor CPU has: data can flow directly between adjacent tiles without going off chip. On the XDNA fabric, compute tiles read each other's local memory and pass data through cascade connections, so a large operand can be distributed across several tiles' SRAM and consumed as a spatial pipeline, the on-chip analogue of a deep software pipeline (AMD, 2024b,a). This is why the **npu** can be extraordinarily efficient for a fixed, well-understood workload—the whole computation is laid out in space and clocked through—and why it is inflexible for anything the layout did not anticipate. The constraint is that the spatial plan is fixed at compile time, so the compiler bears full responsibility for balancing the stages across tiles.

The **static** nature also changes what "optimization" means on an **npu**. On a GPU the compiler searches a large schedule space and the hardware smooths over imperfect choices with dynamic scheduling; on an **npu** there is no smoothing, so a poorly balanced tile assignment idles whole tiles and the compiler must get the balance right rather than approximately right (AMD, 2024b; Wu *et al.*, 2024). This places a heavier burden on the compiler's hardware **model** for the **npu** than for the GPU, because the cost of a mistake is higher and there is no runtime to hide it. For **edge** deployment that determinism is the feature—predictable per-token latency and power—bought with compile-time rigor.

Multi-hardware delta. Where a GPU schedule can be dynamic and an **edge** **npu** schedule must be **static**, the same logical decoder layer demands fundamentally different compilation: the GPU compiler searches schedules and captures launches, while the **npu** compiler assigns tiles and DMA chains at build time with no runtime freedom (AMD, 2024b; Wu *et al.*, 2024). A compiler that targets both must carry both models, which is exactly the per-target specialization this book argues for (Chen and YiRage Contributors, 2026).

1.6 Hardware-aware compilation

Hardware-aware compilation is the discipline of making these constraints first-class compiler inputs rather than afterthoughts, and it reduces to a few recurring decisions: **tiling**, **layout**, and **pass** ordering. **Tiling** chooses the shape of the on-chip working set so it fits the target's fast tier—228 KB shared memory on a GPU, a **cache** level on a CPU, 64 KB SRAM on an NPU—which is the single decision that most directly sets bytes/token (NVIDIA, 2022; AMD, 2024a). **Layout** chooses how a tensor is arranged in memory so the access pattern is coalesced on a GPU, bank-conflict-free in shared memory, or DMA-friendly on an NPU; a wrong **layout** turns a fast on-chip access into a serialized one (Shah *et al.*, 2024; Lai *et al.*, 2023).

The reason these three decisions are coupled is that they all contend for the same scarce resource: on-chip capacity. A larger **tiling** improves compute efficiency but consumes more fast memory; a richer **layout** (padded for bank-conflict-freedom) costs space; double-buffering for a pipeline doubles a buffer's footprint (NVIDIA, 2022; Shah *et al.*, 2024). The compiler cannot optimize them independently because each one's appetite for SRAM constrains the others, which is why hardware-aware compilation is a joint optimization against the modeled capacity rather than a sequence of local choices. A pass pipeline that fixes **tiling** first and discovers later that the **layout** no longer fits has to backtrack, so the residency budget must inform all three from the start.

Precision is the fourth lever that interacts with the other three. Lowering a tensor from BF16 to FP8 or int8 halves or quarters its bytes, which loosens the **tiling** constraint, changes the optimal **layout**, and can even shift the producer/consumer balance of a kernel (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). A hardware-aware compiler therefore co-decides precision with **tiling** and **layout**, because for a memory-bound decode workload the format choice is one of the largest bytes/token levers available—and the hardware **model** must record which precisions each target's compute units support natively.

The third decision, **pass** ordering, is where hardware-awareness becomes subtle. The same set of passes—fusion, **layout** assignment, vectorization—can produce very different code depending on order: **layout** after fusion may force a transpose that re-materializes the activations fusion was meant to keep resident (Lai *et al.*, 2023; Chen *et al.*, 2018). A hardware-aware compiler orders its passes so that the residency-critical decisions are made consistently, and it forks the **pass** pipeline per target because the right order differs by class (Chen, 2020; Chen and YiRage Contributors, 2026). This is why the compiler surveys of Part VI all expose their

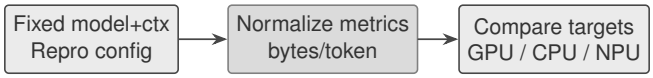


Figure 1.2: Cross-hardware benchmark methodology used in this book.

pass pipelines: the pipeline is where hardware constraints become code.

Review gate. Before merging a hardware-aware change, diff the generated IR or assembly for new buffer allocations, count launches per token on the GPU path and DMA edges on the NPU path, and confirm the **tiling** still fits the target budget (NVIDIA, 2024; Vellaisamy *et al.*, 2026). A change that improves one target can regress another, so the check is per target (Chen, 2026).

1.7 Benchmark methodology

Comparing hardware honestly requires a disciplined **benchmark methodology**, because each vendor's favored **metric** flatters its own silicon. The methodology this book uses, sketched in Figure 1.2, fixes the model and context length, normalizes to decode **metric**s that predict serving cost—bytes/token and launches/token at batch-1—and only then compares targets (Vellaisamy *et al.*, 2026; Chen, 2026). The key discipline is to grade on these decode **metric**s rather than peak FLOPs or an isolated GEMM **benchmark**, because a kernel can win a microbenchmark and lose end-to-end latency when launches and HBM traffic dominate (Davies *et al.*, 2025; Kwon *et al.*, 2023).

The decomposition is the heart of the methodology. A single end-to-end ms/token number tells you a kernel is slow but not why; decomposing it into launches/token (the orchestration term) and bytes/token (the bandwidth term) tells you which limiter to attack, and the two demand different fixes—fusion and capture for launches, residency and quantization for bytes (Vellaisamy *et al.*, 2026; Chen, 2026). This is why the book insists on these two **metric**s rather than a single score: they are actionable in a way a microbenchmark is not. A **benchmark** that reports only GB/s or only TFLOPS hides exactly the information a compiler engineer needs to decide what to change.

A trustworthy **benchmark** also demands reproducibility: pinned configurations, recorded inputs, and logged measurements, so a result on one SKU can be compared to another and to a later software version (Goodfellow *et al.*, 2016; Chen, 2026). This is the same transparency standard the book applies to its own numbers through its fact-verification gate, and it is what turns a cross-hardware

comparison from marketing into engineering. The autobooks loop that produced this book is itself an instance: a fixed harness scores every change on the same **metric**, improvements are kept and regressions reverted, and every number is web-verified before it ships (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Without a fixed **methodology**, the GPU-versus-NPU question degenerates into incomparable vendor numbers; with one, it becomes a measurable routing decision of the kind Chapter 21 formalizes.

Multi-hardware delta. The same **benchmark** must report the same **metric** per target, but the mechanism that moves it differs: launches/token on a GPU, **cache** misses on a CPU, DMA edges on an NPU (NVIDIA, 2022; AMD, 2024a). A methodology that normalizes the **metric** while respecting the per-class mechanism is what makes the comparison fair (Vellaisamy *et al.*, 2026).

1.8 YiRage hardware modeling

The book's reference compiler turns this chapter's constraints into an explicit **model**. **YiRage** carries a per-target hardware **model**—a **chip arch** description of each SKU's on-chip capacities, parallelism grains, and scheduling rules—and the residency, role, pipeline, and synchronization passes of Chapters 6–9 are all functions of that **model** (Chen and YiRage Contributors, 2026). Because the constraints are data rather than hard-coded assumptions, the same passes specialize to a 228 KB H100 SM, a multi-level CPU **cache**, or a 64 KB XDNA tile by reading the **model** for the target (NVIDIA, 2022; AMD, 2024a).

Making the **chip arch** a **model** rather than a constant is what lets the compiler check legality before codegen. A fusion's peak footprint is compared against the modeled tier capacity; a schedule's launch pattern is checked against the modeled orchestration cost; a tile shape is validated against the modeled SRAM and bank structure (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). When a new SKU arrives, updating its **model** re-specializes every pass without rewriting them, which is the scalability property that makes heterogeneous deployment tractable (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). This is the deepest sense in which this book treats hardware as a compiler input: the architecture is a **model** the compiler consults, and the quality of that **model** bounds the quality of every downstream decision.

The **model** also has to capture the cost relationships, not just the capacities. Knowing an H100 SM has 228 KB of shared memory is necessary but not sufficient; the compiler also needs the relative latencies and bandwidths of the tiers, the

launch-floor cost, and the throughput of the **tensor core** relative to the special-function units, because those ratios decide where overlap is worth scheduling (NVIDIA, 2022; Shah *et al.*, 2024). A **chip arch model** that records capacities but not costs can check legality but cannot rank schedules, so a useful **model** carries both—the hard constraints that gate a fusion and the soft costs that order the alternatives (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).

Finally, the **model** is what makes the book's claims portable across SKUs that did not exist when a chapter was written. Because every pass reads the **model** rather than hard-coding "H100," a next-generation GPU with a larger register file or a new **tensor core** instruction is absorbed by updating its **model** entry, and the residency, role, pipeline, and sync passes re-specialize automatically (Chen and YiRage Contributors, 2026; IREE / iree-org contributors, 2024). This is the architectural payoff of treating hardware as data: the compiler's knowledge of silicon lives in one place that can be extended, audited, and verified, rather than scattered through hand-written kernels that rot as hardware moves (Davies *et al.*, 2025).

1.9 Key Takeaways

- **Reason about hardware concretely, not abstractly.** "Fast memory" is useless; "228 KB shared, 256 KB registers, 50 MB L2 on an H100 SM" is a constraint a tiling pass solves against (NVIDIA, 2022; SemiAnalysis, 2024). The difference between a kernel that fits and one that spills is kilobytes on a specific SKU (Vellaisamy *et al.*, 2026).

- **The three classes have different residency and scheduling models.** GPU shared memory and warps (dynamic, launch-taxed), CPU caches and SIMD (coherent, NUMA-bound), NPU tile SRAM and spatial dataflow (static, deterministic) (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b). One schedule is not optimal across them (AMD, 2024a).

- **Decode is bandwidth- and orchestration-bound on every class.** At batch-1 the limiter is bytes through the memory hierarchy and launches per token, not FLOPs, so illegal tier placement—spilling to HBM/DDR—is the recurring failure (Vellaisamy *et al.*, 2026; Chen, 2026). Gate merges on residency, not microbench FLOPs (Davies *et al.*, 2025).

- **Hardware-aware compilation is tiling, layout, and pass order.** These three decisions set bytes/token and must be made per target and in a

residency-consistent order; a wrong layout or pass order re-materializes activations fusion meant to keep on-chip (Lai *et al.*, 2023; Shah *et al.*, 2024). Fork the pipeline per target (Chen, 2020).

- **Model the chip so passes can specialize and check.** YiRage carries a per-target hardware model, so the residency/role/pipeline/sync passes specialize by reading it and verify legality before codegen (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). A new SKU is a model update, not a rewrite (IREE / iree-org contributors, 2024).

1.10 Conclusion

This chapter supplied the concrete hardware constraints that the rest of the book compiles against: the GPU's programmer-managed shared memory and registers with their launch tax, the CPU's hardware caches and NUMA, and the NPU's static tile SRAM and spatial dataflow, each with the exact sizes and grains a decode kernel must respect (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024a). The unifying lesson is that decode is bandwidth- and orchestration-bound on all three, so hardware-aware compilation—tiling, layout, and pass order chosen per target against a concrete model—is what turns the diagnosis of Chapter 1 into shippable kernels (Vellaisamy *et al.*, 2026; Chen, 2026). YiRage makes the architecture an explicit model its passes consult and check, which is how the per-target guarantees of later chapters scale across a fleet (Chen and YiRage Contributors, 2026). With the constraint model established in the abstract, Chapter **??** descends into one class in depth—the CUDA Hopper/Blackwell GPU—mapping its registers, shared memory, TMA, clusters, and tensor cores to the exact optimization bounds a decode kernel must respect (NVIDIA, 2022; SemiAnalysis, 2024).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 2, 3, 4, 7, 8, 9, 11, 12, 13

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 2, 3, 4, 6, 7, 8, 12

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 4, 5, 6, 7, 8, 10, 12, 13

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 2, 9

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 9, 13

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 3, 4, 6, 8, 9, 11, 12, 13

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 8

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 4, 6, 10, 11, 12

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 3, 6, 7, 9, 10, 12, 13

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. `https://iree.dev`. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; `.vmfb` fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 11, 12, 13

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 2, 4, 7, 10

Lai, R., Shao, J., Feng, S., Jin, Y., *et al.* (2023). Relax: Composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103*. TVM Unity; cross-level IRModule (Relax graph + TensorIR), first-class symbolic shapes, call_tir DPS; replaces Relay. 4, 9, 13

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. `https://github.com/mirage-project/mirage`. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; 1.2–6.7× latency reduction. 2, 6

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 2, 3, 5, 9, 11, 12, 13

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 6, 10

Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 2, 3, 6, 8

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 3, 4, 6, 12, 13

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*. Hopper warp specialization (producer TMA / consumer WGMMA), persistent kernels with tile scheduler, pingpong scheduling. 5, 9, 12, 13

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 4, 5, 6, 10, 11, 12, 13

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μGraph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 8, 11, 12, 13

Index

Compiler constraints, 1

Hardware architecture, 1

Memory hierarchy, 1